

PROGRAMMER MANUAL

หุ่นยนต์ส่งเอกสาร

ROBOT MESSENGER

นางสาวมิ่งมานัส ศิวรักษ์

นายศิริชัย วิชชาวุธ

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต

ภาควิชาวิศวกรรมคอมพิวเตอร์

คณะวิศวกรรมศาสตร์

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

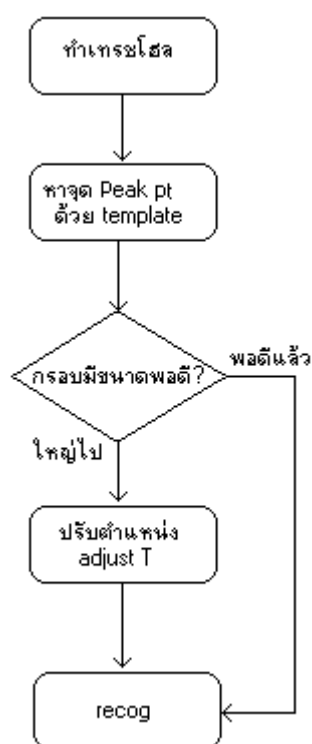
ปีการศึกษา 2545

Programmer Manual

ไฟล์ทั้งหมดเขียนขึ้นด้วย Microsoft Visual C++ โดยมีฟังก์ชันต่างๆดังนี้

การจดจำหมายเลขห้อง (file combine.cpp)

การจดจำหมายเลขห้องได้แบ่งกันเป็นส่วนประกอบต่างๆดังนี้



flow chart รวมทั้งหมด

การหาจุด PeakPt โดยผ่านฟังก์ชัน Template()

Input

CRect big: กรอบล้อมตัวเลข

C2Image Nor2a: ภาพthresholdสูง

Int SumX[]: จำนวนพิกเซลรวมในแต่ละแถวในแนวนอน X

Int SumY[]: จำนวนพิกเซลรวมในแต่ละแถวในแนวนอน Y

output

CRect big: กรอบล้อมตัวเลขที่ปรับปรุงแล้ว

CPoint (return value) จุดบอกตำแหน่งสูงสุดที่ได้จากการทำ matching



การทำ matching ทีละจุดจนหมดทั้งภาพ

การ matching ทีละจุดจะทำการนำภาพ template ไปเทียบทุกจุดที่มี center เป็นสัดำใช้การเทียบทีละ pixel ของภาพที่นำไปเทียบกับภาพใหญ่ที่มี

ได้จุดสูงสุดของภาพ

เป็นจุดที่ได้ผลรวมของค่าแห่ง pixel ที่ได้มากที่สุดเป็นจุดสูงสุด

การปรับกรอบของภาพ

โดยนำจุดสูงสุดที่ได้มาปรับขนาดไปทาง ซ้ายขวา-บนล่าง โดยประมาณจากขนาดความกว้างยาว-สูงของตัวเลข โดยนำไปเทียบกับกรอบของภาพ เทรชโฮลสูง

โดยเปรียบเทียบกรอบว่าใช้กรอบใดเล็กกว่ากัน โดยจะเลือกกรอบนั้นในการล้อม

กรอบด้านบน:ระหว่าง

กรอบบนของเทรชโฮลสูง -20 : จากจุด max Y ที่ได้ -100

กรอบด้านล่าง:ระหว่าง

กรอบล่างของเทรชโฮลสูง +20 : จากจุด max Y ที่ได้ +120

กรอบด้านซ้าย: ระหว่าง

กรอบซ้ายของเทรชโฮลสูง -10 : จากจุด max Y ที่ได้ -100

กรอบด้านขวา:ระหว่าง

กรอบขวาของเทรชโฮลสูง +10 : จากจุด max Y ที่ได้ +160

การปรับตำแหน่งโดยใช้ฟังก์ชัน AdjustT()

Input พารามิเตอร์ที่ส่งเข้ามา

CRect big : กรอบล้อมที่ได้มาจากการฟังก์ชัน Template

CImage Nor: ภาพเทรชโฮลดำ

Int maxX: ค่า x จากจุดmaxPtที่ได้จากฟังก์ชัน Template

Int maxY: ค่า y จากจุดmaxPtที่ได้จากฟังก์ชัน Template

Output CRect big : กรอบล้อมที่ปรับปรุงใหม่

CImage Nor: ภาพเทรชโฮลดำที่ปรับปรุงใหม่



การทำ sum X,Y

เป็นการหาค่าผลรวมของจุดสีดำในแนวแกน X และ Y เก็บไว้ในตัวแปร sumX[] และ sumY[]

แบ่งกรอบเป็น 2 sections

โดยใช้จุด maxY เป็นจุดแบ่ง 2 ส่วนเนื่องจากตัวเลขเรียงขึ้นดังนั้น มีการปรับตำแหน่งที่ไม่เหมือนกันในด้านหน้า-หลัง โดยจะใช้การแบ่ง บน-ล่างเข้ามาช่วย

ปรับครึ่งบน

ครึ่งบนจะเป็นการไล่ตรวจจากจุดที่เป็น maxY ไปจนถึงด้านบนทีละแถว เนื่องจากทางด้านหน้า maxX+40 ตัวเลขจะต่ำกว่า maxY ลงไปดังนั้นจุดที่พบจุดแรกต้องอยู่ห่าง จาก maxY ขึ้นไปไม่มาก(ใน

ที่นี้กำหนดให้ไม่เกิน 15) รวมทั้งจุดที่ห่างกันในแต่ละจุดไม่มาก (ให้เป็น 20) ในขณะที่ ด้านหลัง $\text{maxX}+40$ จุดที่พบจุดแรกก็ไม่ควรสูงไปมากหรือเกิน 1 ตัวอักษร(ประมาณ 70 pixels)

ปรับครึ่งล่าง

ครึ่งล่างจะเป็นการไล่ตรวจจากจุดที่เป็น maxY ไปจนสุดด้านล่างทีละแถว เนื่องจากทาง ส่วน ด้านหลังซึ่งจะเอียงขึ้นจะมีจุดต่ำกว่า maxY ไม่เกิน 40 ด้านหน้า $\text{maxX}+40$ จะมีจุดแรกห่างจาก maxY ลงไปไม่เกิน 75 แต่จะมีจุดห่างลงไปในกรณีที่พบจุดแรกแล้วไม่เกิน 80 pixels

โดยถ้าภาพที่ได้มีจุดใดที่ไม่ตรงกับเงื่อนไขที่กำหนดก็ให้ลบจุดนั้นทิ้ง โดยผลที่ได้จะกลับ นำไปใช้ต่อได้เนื่องจากเป็น พารามิเตอร์ที่ส่งมาแบบ by reference

การจดจำรูปแบบตัวอักษรโดยใช้ ฟังก์ชัน Recog ()

Input

CRect big: กรอบล้อมตัวเลข

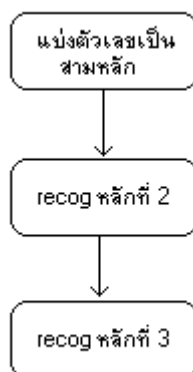
C2Image Source: ภาพ threshold ดำ

Int SumX[]:จำนวนพิกเซลรวมในแต่ละแถวในแนวนอน X

Int SumY[]:จำนวนพิกเซลรวมในแต่ละแถวในแนวแกน Y

Output

C2Image Source: ภาพ threshold ดำที่ปรับปรุงใหม่เพื่อไว้ที่จะเก็บรูปไว้ดูใน file ได้



แบ่งตัวเลขเป็น 3 หลัก



หารความกว้างตามแนวแกน x ด้วย 3

หาจุดต่ำในระยะเวลา ห่างไม่เกิน 5 พิกเซลในแนวแกน Y

ปรับขอบให้พอดีกับหัวท้ายของตัวเลข

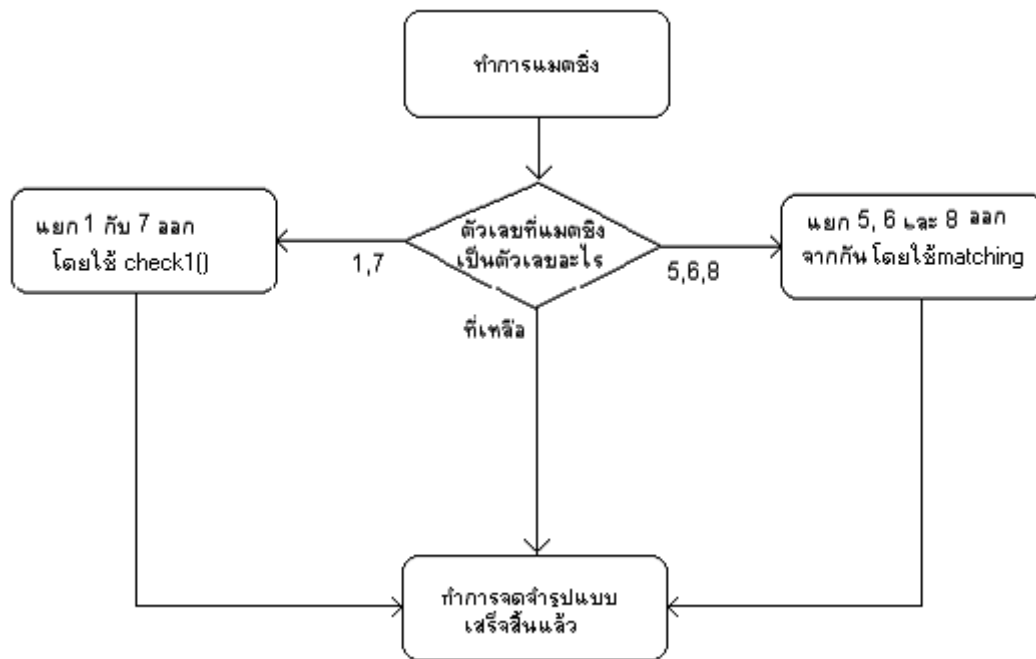
การปรับขอบหัวท้ายให้พอดีโดยการเลื่อนตำแหน่งของช่วงตัวเลขที่มีผลรวมของจำนวนพิกเซลเท่ากับตำแหน่งนั้นไปเรื่อยๆ ถ้าเป็นตำแหน่งหัวก็เลื่อนไปทางขวา ถ้าตำแหน่งท้ายก็เลื่อนไปทางซ้าย

การ recog ตำแหน่งที่ 2

ส่วนตำแหน่งที่ 2 มีแค่ตัวเลข 0 และ 1 จึงใช้เพียงแค่ check1 อย่างเดียวก็พอ

การ recog ตำแหน่งที่ 3

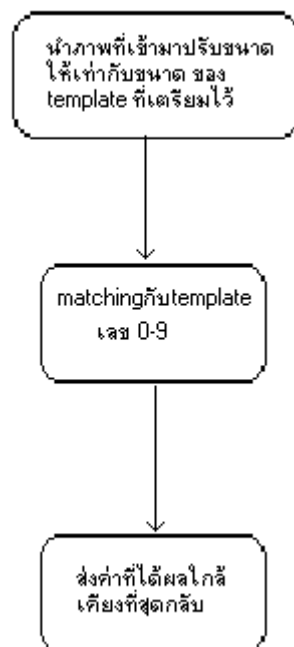
ในตำแหน่งที่ 3 มีทั้งตัวเลข 0-9 ดังนั้นจะต้อง ตรวจสอบทุกตัวเลข ตาม โพลัวชาร์ตด้านล่าง



ฟังก์ชัน matching()

input C2Image no: ภาพส่วนของตัวเลขที่ส่งมาเป็นรูปภาพเพื่อทำการ matching

Output return เป็น int :ตัวเลขที่ใกล้เคียงที่สุด เป็นตัวเลข 0-9



ฟังก์ชัน check1()

input CRect no: กรอบล้อมรอบรูป สีเหลี่ยม

int pre: ตำแหน่งซ้ายที่สุดของตัวเลขก่อนหน้า

int aft: ตัวเลขก่อนหน้าเป็น 1 รีเพล

Output int: "1" ถ้าตรวจแล้วพบว่าเป็นเลข 1

"0" ถ้าตรวจแล้วพบว่าเป็นเลข 0

ถ้าเป็นเลข 1 จะมีคุณสมบัติ ดังต่อไปนี้

- มีความกว้างไม่เกิน 30 พิกเซล
- มีความสูงไม่เกิน 50 พิกเซล
- ห่างจากเลขก่อนหน้ามากกว่า 7 พิกเซล และ ถ้าข้างหน้าเป็นเลข 1 จะห่างจากเลขก่อนหน้าอีกเพิ่มขึ้น 5 พิกเซล

```
int check1(CRect no,int pre,int aft){    //check if this is 1
/* Parameter 1.no,rectangle that cover that number
    2. the leftmost of the previous number
    and
    3. aft,if previous number is 1 then aft is 1(default is 0)
*/
    if( no.left -pre>=(aft*5)+7 &&no.Size ().cx<30&&no.Size().cy<50)
        return 1;
    return 0;
}
```


ฟังก์ชัน Strobes()

Input CRect bound :ตำแหน่งที่ล้อมรอบตัวเลขที่จะมาทำ strobe

C2Image pic :รูปของเทรซโฮลต่ำที่จะส่งมาทำ strobe

int size :ขนาดของการทำ strobe ในที่นี้จะเป็น 4 คือ 4 x 4

Output int :ผลที่ได้ว่าเป็นตัวเลข 5 8 หรือ 9



หา strobes จากข้างหน้า

คือการนับสโตรปจากซ้ายไปขวา ดังแสดงในทฤษฎีของปริณูณิพนธ์

หา strobes จากข้างหลัง

คือการนับสโตรปจากขวาไปซ้าย ตรงกันข้ามกับข้างบน

หาความแตกต่าง ของ 5,8,9 (ใช้ฟังก์ชัน matching16)

หาความแตกต่างโดยใช้

- ตำแหน่งล่างขวาแยก 8 กับ 5 และ 9
- ตำแหน่งบนซ้ายแยก 5 และ 9

ฟังก์ชัน slip()

Input int x: ตำแหน่งของเส้นแบ่งตัวเลขเก่า ที่แบ่งโดยการนำขนาดทั้งหมดมาหารเฉลี่ย

int sumX[]: จำนวนพิกเซลรวมในแต่ละแถวในแนวแกน X

int right: ทิศทางของการเลื่อน หากเท่ากับ “1” จะเลื่อนไปทางขวา นอกนั้นทางซ้าย

Output int: ตำแหน่งใหม่ของเส้นแบ่งตัวเลข

```

int slip(int x, int sumX[], int right){
    int i,stop=x+5;
    int lowest=sumX[x];
    int lowestPos=x;
    bool flag=true;

    ////Find lowest pos
    for(i=x-10;i<=x+10;i++){
        if(sumX[i]<lowest ){
            lowest=sumX[i];
            lowestPos=i;
        }
    }

    ///Adjust the position
    if(right) { //step right
        i=lowestPos+1;
        while (sumX[i]<=lowest) {
            lowestPos=i;
            i++;
        }
    }else{ //step left
        i=lowestPos-1;
        while (sumX[i]<=lowest) {
            lowestPos=i;
            i--;
        }
    }

    return lowestPos;
}

```